
How to Build a Framework for Systematic Trading

A complete, step-by-step architecture for turning ideas into rules, testing them honestly, and running them live — with an education-only disclaimer.

PUBLISHED BY STRATLAB

Educational content only — not investment advice. StratLab is not SEBI-registered.

How to Build a Framework for Systematic Trading

A complete, step-by-step architecture for turning ideas into rules, testing them honestly, and running them live — with an education-only disclaimer.

Published by StratLab

Important Disclaimer

This book is provided for **educational purposes only**. It is not investment advice, a recommendation, or a solicitation to buy or sell any security.

- **StratLab is not registered with SEBI** as an investment adviser, research analyst, or portfolio manager, and does not provide investment advisory services.
- Trading and investing involve substantial risk of loss, including the possible loss of principal. Past performance never guarantees future results.
- All examples, figures, and historical observations are illustrative. A framework, however well-built, cannot guarantee profitability — it can only make your process *systematic* and your testing *honest*.
- Data, costs, taxes, and regulatory details change over time. Always verify current rules and consult a qualified adviser before acting.
- Nothing in this book should be read as a promise of return or a guarantee that any strategy will be profitable.

By reading this book, you agree that you are solely responsible for your own trading and investment decisions and that you will consult a SEBI-registered adviser where appropriate.

How to Read This Book

This book is about **how** to build a systematic trading operation — the *framework*, not any single strategy. It is the infrastructure that the strategies in the other books in this series (Quant Trading, Risk Management, Momentum, and the rest) plug into.

- **Chapters 1–2** define systematic trading and map the full lifecycle — the end-to-end process from idea to live and back.
- **Chapters 3–6** build the core: turning an idea into a testable hypothesis, sourcing clean data, generating signals, and constructing a portfolio.
- **Chapters 7–9** cover the disciplines that separate working systems from broken ones: honest backtesting, execution and monitoring, and risk/governance.
- **Chapter 10** assembles everything into a written framework template.
- The **appendix** is a lifecycle checklist you can run against any system you build.

A note on language: this book describes *how* to build and test systems — never what you "should" trade. Where behaviour is discussed, it is framed as historical observation, not promise. You are responsible for your own decisions.

Table of Contents

1. What Is Systematic Trading?
 2. The Framework Lifecycle
 3. From Idea to Testable Hypothesis
 4. Data: Sourcing, Cleaning, and Integrity
 5. Signal Generation
 6. Portfolio Construction and Position Sizing
 7. Backtesting Done Right
 8. Execution, Costs, and Monitoring
 9. Risk Management and Governance
 10. A Working Framework Template
- Appendix — Lifecycle Checklist
-

1. What Is Systematic Trading?

A process, not a strategy

Systematic trading is the practice of making trading decisions through *explicit, pre-defined rules* rather than *in-the-moment judgement*. The defining feature is not the use of code — it is that the rules can be **written down, tested, and repeated** identically, by anyone, at any time.

This book is about building the **framework** — the repeatable process — that surrounds those rules. A framework is not a strategy; it is the *machine that turns ideas into strategies, tests them, runs them, and retires them*.

The rules-versus-judgement line

The difference between systematic and discretionary is a line, not a wall:

- **Fully systematic** — every decision (what, when, how much, when to exit) is a rule. A computer (or a checklist) could execute it.
- **Quant-assisted** — a human decides, but with systematic inputs (a screen, a risk model, a backtest).
- **Discretionary** — judgement drives everything.

A systematic *framework* can support all three; its heart is the discipline of **explicitness**. If a decision cannot be written as a rule, it cannot be tested — and the framework exists precisely to make decisions testable.

The five promises of a framework

Why build one at all? A framework delivers five things no ad-hoc approach can:

1. **Testability** — rules can be backtested. You can learn how an idea *would have behaved* before risking money.
2. **Repeatability** — the same inputs produce the same decisions, every time, regardless of mood.
3. **Honesty** — a framework forces you to account for costs, drawdowns, and failure modes that discretionary trading hand-waves away.
4. **Scale** — a systematic process can run across hundreds of instruments and thousands of decisions.
5. **Improvability** — because every decision is recorded and every rule is explicit, the system can be *measured* and *improved* — which is impossible for a process that lives only in someone's head.

What a framework is built from

A complete systematic framework has **five components**, each the subject of a chapter in this book:

1. **Hypothesis** — the idea, stated as a testable claim (Chapter 3).
2. **Data** — the clean, complete, point-in-time inputs (Chapter 4).
3. **Signals** — the rules that say *what* and *when* (Chapter 5).
4. **Portfolio construction** — the rules that say *how much* (Chapter 6).
5. **Execution, monitoring, and risk** — the rules that keep the system honest and alive (Chapters 7–9).

Strip any framework down and it is these five boxes, connected by a lifecycle (next chapter).

The honest caveats

Three honest limits, stated up front:

1. **A framework does not create edge.** It *tests for* edge and *discipline* it — but the edge itself comes from the *idea*. A great framework around a bad idea still loses money.
2. **A framework is only as honest as you let it be.** The framework's whole purpose is to *expose* failure; if you use it to *rationalise* (tweak until the backtest looks good), it becomes a tool for self-deception.
3. **The framework is not the point; survival is.** The framework exists to keep you in the game long enough for a real edge to compound. Everything in it serves that one goal.

Summary

- Systematic trading = explicit, testable, repeatable rules.
- A framework is the process that surrounds the rules, not a strategy itself.
- It delivers testability, repeatability, honesty, scale, and improvability.
- Five components: hypothesis, data, signals, portfolio construction, execution/monitoring/risk.
- It does not create edge; it tests for it and disciplines it.

Next: the lifecycle — how the five components connect into an end-to-end process.

2. The Framework Lifecycle

The machine, end to end

A systematic framework is best understood as a **cycle** — a loop that an idea travels, again and again. Understanding the loop is more important than any single component, because most failures are failures to *complete the loop* — to carry an idea all the way around, honestly, and then to feed what is learned back in.

The eight-stage loop

1. Idea
2. Hypothesis (a testable claim)
3. Data
4. Signal
5. Backtest
6. Paper trade
7. Live (small → scaled)
8. Review & retire (→ back to 1)

1. Idea

A *why-based* claim about a pattern: "investors under-react, so winners keep winning." The idea is the raw material — necessary, but not yet testable.

2. Hypothesis

The idea, made **falsifiable**: "ranking stocks by 12-1 return and holding the top decile, rebalanced monthly, produces positive risk-adjusted returns after costs." A hypothesis is a claim that can be *shown wrong* — and a good framework *tries* to show it wrong.

3. Data

The clean, complete, **point-in-time** inputs (Chapter 4). This is where most retail systems quietly fail: survivorship bias, look-ahead, and corporate-action errors corrupt everything downstream.

4. Signal

The rule that turns data into a *decision*: what to hold, when to enter, when to exit (Chapter 5).

5. Backtest

Running the signal over history and *attacking the result* (Chapter 7). The backtest is an attempt to **disprove**, not prove.

6. Paper trade

Running the system live, in real time, with no (or tiny) money — to test the *engineering*: execution, fills, costs, and the daily operational discipline.

7. Live

Going live **small**, then scaling only after live results *track* the backtest and paper results. Live is where the tail events and the psychology (Risk Management book) show up.

8. Review & retire

Measuring live against expectation, watching for **edge decay**, and *retiring* the system when it stops behaving as it should — then feeding the lessons back into the next idea.

The two properties that make it work

Each stage is a gate

A stage is not a formality; it is a **kill-point**. An idea that fails the hypothesis does not proceed; a strategy whose paper trading diverges from its backtest does not go live; a live system that stops tracking is retired. The stages exist to **kill bad ideas cheaply** — the earlier, the cheaper. The single most common failure is *skipping a stage* — going from idea straight to money, or trusting an unattacked backtest.

The loop closes

The final stage feeds back into the first. Every retired system teaches something — a regime where momentum fails, a cost you under-estimated, a data error — and that lesson becomes the seed of the *next* idea. A framework is not a pipeline that ends; it is a loop that *compounds* learning.

A mental model to carry

Think of the framework as a **quality filter**: it takes in raw ideas and, at each stage, lets through only the fraction that survive honest scrutiny. The fraction that reaches live is small — and that is the point. The framework's value is not in *finding* ideas; it is in **not letting bad ideas through**.

Summary

- The loop: idea → hypothesis → data → signal → backtest → paper → live → review/retire → back to idea.
- Each stage is a gate; the loop kills bad ideas cheaply, early.
- The loop closes: every retirement feeds the next idea.

- The framework is a quality filter, not an idea-generator.

Next: the first real stage — turning an idea into a testable hypothesis.

3. From Idea to Testable Hypothesis

The difference between a hunch and a claim

Most "trading ideas" are actually **hunches** — vague, untestable, and unfalsifiable. The first job of a framework is to convert a hunch into a **hypothesis**: a specific, falsifiable claim that can be tested and, crucially, *shown to be wrong*.

This chapter covers that conversion.

What makes a claim testable

A testable hypothesis has four properties. It states:

1. **The mechanism** — *why* the pattern should exist (not just *that* it appeared in a backtest).
2. **The exact rule** — the precise, unambiguous signal.
3. **The measurable claim** — what "working" would look like (a metric, a threshold, a benchmark).
4. **The falsification** — what would prove it *wrong*.

A weak idea (a hunch)

"Momentum seems to work."

Untestable: no mechanism, no rule, no measure, no falsification. A framework cannot do anything with this.

A strong idea (a hypothesis)

"Because investors under-react to information (mechanism), ranking NIFTY 500 stocks by their 12-month return skipping the last month and holding the top decile, rebalanced monthly (rule), produces a positive annualised return after costs (measure) that is higher than the NIFTY 50 benchmark over the same period (falsification target)."

Every word can be checked. This is what a framework consumes.

The mechanism test

The single most important part is the **mechanism** — the "why". A hypothesis with a mechanism is *robust*: if the mechanism is a real, persistent feature of markets, the edge is likely to persist. A hypothesis without a mechanism is just a pattern — and a pattern with no explanation is the classic signature of overfitting.

The test is simple: **can you explain the edge in a sentence, in terms of human behaviour or market structure?**

- *"Investors under-react to news"* → a mechanism (behavioural).
- *"Funds rebalance at month-end"* → a mechanism (structural).
- *"It worked in my backtest"* → **not** a mechanism.

If you cannot articulate the mechanism, the idea is not ready.

Stating the rule precisely

The rule must be **unambiguous** — so precise that two different people would implement it identically:

- *Universe*: exactly which instruments, with what liquidity filter.
- *Signal*: the exact condition, in the exact words.
- *Sizing*: how much, and by what rule.
- *Exit/rebalance*: when, and on what trigger.

Ambiguity is where look-ahead bias and hindsight creep in. Every vague phrase ("strong momentum", "cheap", "recently") must be replaced with a number and a window.

Pre-registering the test

The most powerful discipline in systematic trading is **pre-registration**: write down the hypothesis, the rule, and the success/failure criteria *before* you run the backtest. This matters because after you see the backtest, you will be tempted to tweak — and every post-hoc tweak is a step toward overfitting.

Pre-registration forces the clean sequence: **claim** → **test** → **verdict**. Not: test → claim → tweak → test → claim.

A hypothesis template

```
Mechanism: [why the edge should exist]
Universe:  [exactly what is traded, and the liquidity filter]
Signal:    [the exact entry/exit rule]
Sizing:    [how much, by what rule]
Claim:     [the measurable outcome: metric + benchmark + horizon]
Falsify:   [what result would prove it wrong]
```

Fill this in — honestly, before testing — and you have converted a hunch into something a framework can actually work with.

Summary

- A testable hypothesis has a mechanism, an exact rule, a measurable claim, and a falsification condition.
- The mechanism ("why") is the most important part; no mechanism → likely overfit.
- State the rule unambiguously; vagueness invites hindsight bias.
- Pre-register the test before running it, to avoid post-hoc tweaking.
- Use the template: mechanism, universe, signal, sizing, claim, falsify.

Next: data — where most systems silently fail.

4. Data: Sourcing, Cleaning, and Integrity

Where most systems silently fail

A systematic framework is only as honest as its data. Every stage downstream — signal, backtest, live — inherits whatever is wrong upstream. This chapter covers the three data disciplines: **sourcing**, **cleaning**, and **integrity**.

The three data requirements

Before any analysis, your data must be:

1. **Clean** — free of errors, gaps, and bad ticks.
2. **Aligned** — prices adjusted correctly for corporate actions (splits, bonuses, rights, dividends).
3. **Point-in-time** — containing only the information that was *actually available* on each historical date.

The first two are about correctness; the third is about **not cheating** (look-ahead bias).

Sourcing

For Indian equities, the data landscape is roughly:

- **Price/volume data** — daily OHLCV for NSE/BSE equities and indices, from your broker's API, a commercial data vendor, or exchange-derived sources.
- **Fundamental data** — financial statements and ratios; *point-in-time* fundamentals are much harder to obtain than current snapshots, and this gap matters.
- **Corporate actions** — splits, bonuses, rights issues, dividends; essential for correct adjustment.

The sourcing rule: **prefer data that is documented, auditable, and point-in-time** over data that is convenient but opaque. A beautiful backtest on bad data is worthless.

Cleaning

Cleaning catches the errors that corrupt results:

- **Duplicate or missing bars** — a stock with a 10-day gap in its history will produce garbage momentum signals there.
- **Bad ticks** — a spurious price of ₹0.01 or ₹99,999 that is a data error, not a trade.
- **Zero or negative prices** — nonsense values that must be removed (and their cause understood).

The rule: **never silently drop data**. When you clean, record *what* you dropped and *why* — a data-removal log is an audit trail, and it is the only defence against the accusation (including your own) that you deleted data to make the backtest look better.

Alignment (corporate actions)

This is the subtle one. A stock that did a 1:1 bonus (a 2-for-1 split) shows a price that *halved overnight* — but nothing about the company changed. If your data is not adjusted:

- A momentum signal will see a phantom −50% move and mis-rank the stock.
- A backtest will record a phantom −50% loss (or gain) that never happened to the holder.

The rule: **use adjusted prices** (adjusted for splits, bonuses, and rights) for *all* return calculations — and understand exactly what your vendor's "adjusted" means (some adjust for dividends too, which changes total return).

Point-in-time integrity

This is where look-ahead bias lives. Three classic violations:

1. **Survivorship bias** — using today's universe (the companies that *survived*) to backtest the past. The delisted, merged, and bankrupt names are missing — and their absence flatters every result. Use a **point-in-time universe**: the actual tradeable list on each date.
2. **Restated fundamentals** — using today's *restated* earnings to rank stocks *last year*. Last year, the market only knew last year's (pre-restatement) numbers. Using restated data is look-ahead.
3. **Future corporate actions** — using a split-adjusted price on a date *before* the split happened.

The universal test: **for every data point, ask "did the market actually know this on this date?"** If not, it must not enter the signal.

The discipline

Data integrity is a *process*, not a one-time task:

- **Document the source** and its known biases (every vendor has some).
- **Log every cleaning step** — what was dropped and why.
- **Use point-in-time universes and un-restated fundamentals** for anything historical.
- **Reconcile live vs. historical data** (Chapter 8) — if they disagree on the same day, find out why before trusting either.

Summary

- Data must be clean, aligned, and point-in-time.

- Prefer documented, auditable, point-in-time sources.
- Clean with a log — never silently drop data.
- Use corporate-action-adjusted prices for all returns.
- The universal test: "did the market know this on this date?"

Next: signal generation — turning data into decisions.

5. Signal Generation

Turning data into decisions

The **signal** is the heart of the system: the rule that turns clean data into a *decision* — what to hold, when to enter, when to exit. This chapter covers how to design a signal that is precise, robust, and honest.

The anatomy of a signal

Every signal, however complex, answers three questions in a specific order:

1. **What** — which instruments are eligible? (a *filter*, not a timing decision.)
2. **When** — under what condition do we act? (the *entry* trigger.)
3. **When to leave** — under what condition do we exit? (the *exit* rule.)

A complete signal specifies all three. A signal that only says *what* to buy, with no exit, is half a system.

The three families of signals

Signals fall into a small number of families, and knowing the family tells you how it will behave:

- **Momentum / trend** — buy strength, ride the trend, exit on weakness. Low win rate, large winners, suffers in ranges.
- **Mean reversion** — fade extremes, buy dips, exit on the snap-back. High win rate, occasional large loss, suffers in trends.
- **Cross-sectional / relative** — rank a universe and hold the best *relative* to the others. Always invested in something.

Most systems are one of these, or a combination (e.g., a trend filter gating a mean-reversion entry). The companion books in this series explore the specific strategies in each family.

Designing for robustness

A good signal is **robust** — its edge survives small changes to its parameters. The design principles:

1. Few parameters

Every parameter is an opportunity to overfit. A signal with three parameters is testable; one with thirty is curve-fit. The discipline: **explain, in words, why each parameter is there** — and if you cannot, cut it.

2. Economically sensible values

Parameters should be *sensible*, not *optimised*. A 200-day trend filter has a reason (it approximates the long-term trend); a 137-day filter was almost certainly mined from the data. Prefer round, meaningful values (12 months, 200 days) over odd ones that only exist because they backtested well.

3. Insensitivity to small changes

The test of robustness: **does the edge survive if you change each parameter slightly** (10 vs. 12 months, 200 vs. 190 days)? A robust edge has a *flat-topped* performance surface — performance is similar across a neighbourhood of parameter values. A fragile edge is a single sharp spike at one lucky value.

4. A stated mechanism

The signal must have a *why* (Chapter 3's mechanism test). A signal with no economic rationale, however good its backtest, is a coin-flip wearing a spreadsheet.

The exit deserves equal design effort

Most people over-design the entry and under-design the exit. That is backwards, because **the exit is where most of the risk lives**. A complete exit specifies:

- **The stop** — the level at which the thesis is wrong (structural) or the risk is exceeded (volatility-based).
- **The take-profit / target** — where the swing is exhausted (for bounded strategies).
- **The trail** — how open profit is protected (for trend strategies).
- **The time stop** — exiting dead-money trades after a fixed holding period.

An entry without a defined exit is not a strategy; it is a hope.

Pseudocode for a complete signal

```
for each instrument in universe:                # WHAT (filter)
    eligible = liquidity_filter(instrument)        # tradable?

    if eligible:
        state = signal_state(instrument)          # momentum / reversion / relative

        if state == "enter" and no position:      # WHEN (entry)
            open_position(instrument)
            set_stop(instrument)                  # exit rule defined AT entry
            set_target_or_trail(instrument)
```

```
if position and exit_condition(instrument): # WHEN TO LEAVE
    close_position(instrument)
```

Note what the pseudocode makes explicit: **the exit is set at the moment of entry**, and the signal is *complete* — every decision point is covered.

The honest test

A signal is finished when you can hand it to someone else — someone who has never seen the backtest — and they can run it and get the *same* decisions. If you have to explain "well, here I'd override it because...", the signal is not finished; it is discretionary trading wearing a rules costume.

Summary

- A signal answers what, when, and when-to-leave — all three.
- Three families: momentum/trend, mean reversion, cross-sectional/relative.
- Design for robustness: few parameters, sensible values, insensitivity, a mechanism.
- The exit deserves equal design effort — and is set at entry.
- A finished signal is one someone else can run identically, with no overrides.

Next: portfolio construction and position sizing — the "how much" layer.

6. Portfolio Construction and Position Sizing

The "how much" layer

A signal tells you *what* and *when*; portfolio construction tells you **how much** — and *how much* is where a good signal becomes a good (or a ruined) system. This chapter covers the construction and sizing layer, in close partnership with the Risk Management book in this series.

The two decisions

Portfolio construction is two decisions:

1. **How much per position** — the sizing rule (fixed-fraction, volatility, Kelly — see the Risk Management book).
2. **How much in total** — the aggregate exposure, and how it is spread across positions.

Both matter, and they are governed by different logics.

Sizing per position

The universal foundation: **risk a fixed, small fraction of capital per trade** (the 1–2% rule), with size *derived* from the stop:

Position size = $(\text{Equity} \times \text{Risk\%}) \div (\text{Entry} - \text{Stop})$

This equalises risk across trades regardless of stop distance, and — because it scales with equity — it self-stabilises: it compounds in wins and de-risks in losses. Every sizing method (fixed-fraction, fixed-ratio, ATR/volatility, Kelly, fractional-Kelly) is a variation on this one idea; the Risk Management book covers them in depth.

For a *systematic* framework, the key requirement is that **the sizing rule is as explicit as the signal** — written down, computed from the stop, and applied identically every time.

Portfolio construction: how the positions combine

A portfolio is not just a list of sized positions; it is a *structure*, and the structure carries risk of its own:

Equal-weight vs. signal-weight

- **Equal-weight** — every position gets the same allocation. Simple, robust, and the default for most factor and momentum systems.
- **Signal-weight** — positions are sized by the *strength* of the signal (e.g., stronger momentum = larger position). Captures more of the signal, but concentrates risk in the strongest (and often most crowded) names.

For most retail systems, **equal-weight** is the right starting point: it is robust and avoids the hidden concentration of signal-weighting.

Diversification across drivers

Ten positions that are all large-cap banks are one bet, not ten (the Risk Management book's correlation point). Construction must **diversify across drivers** — sectors, sizes, styles, or asset classes — not just across tickers.

The exposure cap

A systematic framework needs a **maximum total exposure** — a cap on how much of the account is deployed, and how much is at-risk in aggregate. The signal may say "buy 15 names"; the construction layer says "but never more than X% of equity in the market, and never more than Y% total at-risk."

Volatility targeting as the portfolio-level control

A particularly useful portfolio-level tool is **volatility targeting**: scale the *entire* portfolio's size so that its expected volatility is roughly constant (see the Quant Trading book). This automatically de-risks the whole book when the market gets wild and re-risks it when it calms — a systemic, rather than per-trade, risk control.

Pseudocode for the construction layer

```
capital = account_equity
risk_per_trade = 1% of capital

for each signal position:
    size = (capital * risk_per_trade) / (entry - stop)
    # cap: single-position size <= max_position_pct * capital

total_at_risk = sum(size_i * (entry_i - stop_i))
if total_at_risk > max_total_risk:
    scale_down_all_positions_to_fit(max_total_risk)

if net_exposure > max_exposure:
    reduce_to_fit(max_exposure)
```

The pseudocode makes the layers visible: per-trade sizing, a per-position cap, an aggregate at-risk cap, and an exposure cap. Each is a *rule*, checked every rebalance.

The discipline

The construction and sizing layer is where **most systems are actually won or lost** — not because sizing is clever, but because sizing is where over-betting happens, and over-betting is how positive-expectancy systems get ruined (the Risk Management book's core argument). The framework's job here is to make over-betting *impossible* by writing the caps into the rules.

Summary

- Two decisions: how much per position, and how much in total.
- Size per position from the stop (1–2% risk); the rule must be as explicit as the signal.
- Prefer equal-weight for robustness; diversify across drivers, not just tickers.
- Cap total at-risk and total exposure; consider volatility targeting at the portfolio level.
- Sizing is where over-betting happens — write the caps into the rules.

Next: backtesting done right — the discipline that keeps the framework honest.

7. Backtesting Done Right

The discipline of trying to disprove

A backtest is the moment of truth — but only if it is run *honestly*. The default posture of a systematic framework must be: **my backtest is lying; my job is to find out how**. This chapter covers the discipline that turns a backtest from a sales pitch into an experiment.

The mindset: test to disprove, not prove

Most people backtest to *confirm* their idea — they want the backtest to say "yes." The result is that they unconsciously tune, filter, and cherry-pick until it does. A framework inverts this: it backtests to **try to kill the idea**, and only ideas that survive honest attack are worth proceeding with.

The practical form of this inversion:

- **Add the costs you wish you could ignore** (they are real live).
- **Remove the survivorship bias** you would rather not see.
- **Stress the parameters** rather than showing only the best one.
- **Test out-of-sample** rather than reporting in-sample.

The attacks, in order

1. Add realistic costs

Brokerage, STT, stamp duty, and — above all — the **bid-ask spread and slippage**. In India, the spread and STT on churning strategies are frequently enough to turn a "profitable" backtest negative. Rule: **run the backtest with pessimistic costs**. If the edge needs optimistic costs to survive, it is fragile.

2. Remove survivorship bias

Backtest on the **point-in-time universe** — the names that actually existed on each date, including those that later delisted or failed (Chapter 4). Today's survivor list flatters every result.

3. Hunt look-ahead bias

For every input, verify it was *known at decision time* (Chapter 4's universal test). Restated fundamentals, adjusted prices used pre-action, and any use of the future are all look-ahead.

4. Stress the parameters

Show performance *across a range* of each parameter, not just the best value. A robust edge has a flat-topped surface; a single sharp spike is the fingerprint of overfitting.

5. Test out-of-sample

The gold standard: fit nothing, or fit on one period and test on a *later, unseen* period. A strategy whose edge persists out-of-sample — without further tweaking — is worth something. One that collapses out-of-sample was overfit.

6. Demand a large sample and regime coverage

A backtest with 20 trades proves little; one with 2,000 is far more credible. And a strategy tested only in a bull market has learned nothing about bear markets. Prefer long histories spanning **multiple regimes**.

Reading the output honestly

When the backtest prints its numbers, read them in the order that matters (from the Quant Trading book's metrics chapter):

1. **Trade count** — is the sample big enough?
2. **Max drawdown** — could I survive it?
3. **Profit factor / expectancy** — is there actually an edge?
4. **Sharpe / Sortino** — is the edge worth the risk?
5. **Robustness** — does it survive the attacks above?

CAGR — the number everyone reads first — is the one most easily inflated by a lucky, overfit, costless backtest. Read it last.

The pre-registration discipline

The single strongest defence against self-deception is **pre-registration** (Chapter 3): commit to the hypothesis, the rule, and the success/failure criteria *before* running the backtest. After you have seen the result, every change you make is a step toward overfitting — and pre-registration is the record that keeps you honest about that.

The honest bottom line

A backtest is not a *prediction* of future returns; it is a **stress test of an idea's robustness**. Its value is negative: it cannot tell you the strategy *will* work, but it can tell you — if you let it — that the strategy *probably won't*. The framework's entire backtesting discipline is built to extract that negative value honestly.

Summary

- Test to disprove, not prove; default to "my backtest is lying."
- Attacks: add costs, remove survivorship, hunt look-ahead, stress parameters, test out-of-sample, demand sample size and regime coverage.
- Read trade count and drawdown before CAGR.
- Pre-register before testing to resist post-hoc tweaking.
- A backtest is a stress test, not a prediction.

Next: execution, costs, and monitoring — the gap between backtest and live.

8. Execution, Costs, and Monitoring

The gap between backtest and live

A strategy that survives an honest backtest can still lose money live — because live differs from the backtest in ways that are easy to forget. This chapter covers the three differences: **execution**, **costs**, and **monitoring**.

Execution: the price you actually get

The backtest assumed you traded at the signal price, instantly. Live, you trade at **the market's price**:

- **Slippage** — the gap between expected and actual fill, especially in fast markets or illiquid names.
- **Latency** — the delay between the signal firing and the order reaching the market.
- **Partial fills** — not getting the whole order at one price.

The discipline: **pad the backtest with realistic slippage**, trade liquid instruments, and — critically — **measure live fills against the backtest's assumption**. If you consistently fill worse than modelled, the edge is smaller than you thought, and the model must be updated to reflect reality.

Costs: larger than you estimated

The backtest estimated costs; live costs are usually **larger**, and they appear in places the backtest ignored:

- A strategy that rebalances at the close pays the **close-auction spread**.
- A strategy that trades illiquid small-caps pays **market impact** — your own order moves the price.
- **Taxes** (STT, capital gains) on realised turnover reduce the net return in ways a costless backtest cannot show.

The discipline: **over-estimate costs in the backtest** (Chapter 7), and reconcile live costs against the estimate. A strategy whose live costs are persistently above the estimate is a strategy with a smaller (or no) edge.

Monitoring: is the system still itself?

A live system must be **monitored**, not just run. The monitoring has three jobs:

1. Track live against backtest

Compare live performance to what the backtest *predicted* for the same period. A persistent gap is a **modelling error** (a cost, a data difference, a look-ahead you missed), not bad luck. Find it.

2. Watch for edge decay

Edges decay — markets adapt, competitors arrive, the anomaly gets arbitrated. The signature is live performance *drifting away* from the backtest's behaviour over time. Monitoring's job is to notice the drift **early**, before it costs real money.

3. Watch for operational failures

The unglamorous but critical work: did the data arrive on time? Did the orders fill? Did the position sizes match the rules? An operational error — a missed rebalance, a duplicated order, a stale price — can cost more than a bad signal. Monitoring is the safety net.

The operational discipline

Running a system is an *operating* job, and it deserves operating discipline:

- **A runbook** — a written procedure for each daily/weekly task (pull data, generate signals, place orders, reconcile).
- **Reconciliation** — every day, compare what *should* have happened (per the rules) against what *did* happen (per the fills and positions). Any difference is investigated, not ignored.
- **A kill switch** — pre-defined conditions under which you stop the system (a drawdown threshold, a tracking-error threshold, a data failure). Decided in advance, not in the panic of the moment.

Summary

- Execution: pad for slippage, trade liquid, and reconcile live fills against the model.
- Costs: over-estimate in the backtest; reconcile live costs against the estimate.
- Monitoring: track live vs. backtest, watch for edge decay, catch operational failures.
- Run with operating discipline: a runbook, daily reconciliation, and a pre-defined kill switch.

Next: risk management and governance — the layer that keeps the system alive.

9. Risk Management and Governance

The layer that keeps the system alive

A systematic framework is a machine, and machines need **governance** — rules about the rules. This chapter covers the risk and governance layer: the discipline that protects the system from its two greatest enemies — **its own over-betting** and **its own operator**.

Risk management: wired into the rules

Risk management is not a separate step; it is **baked into the framework** at every layer (and explored in depth in the Risk Management book). In the framework's terms, risk shows up as:

- **Per-trade risk** — the 1–2% rule, with size derived from the stop (Chapter 6).
- **Portfolio risk** — the cap on total at-risk and total exposure, and diversification across drivers.
- **Leverage/margin discipline** — risk computed on notional, a margin buffer, and a preference for defined-risk structures.
- **Tail risk** — sizing for the gap, avoiding oversized positions through binary events, capping aggregate short-vol exposure.

The framework's contribution is to make these **rules, not intentions** — written down, checked every rebalance, and impossible to override in the heat of the moment.

Governance: rules about the rules

Governance answers a different question: **who decides when to change the system, and under what conditions?** Without governance, the operator is the system's weakest link — they will tweak it mid-drawdown, abandon it at the bottom, and over-bet it when it is working.

Three governance rules matter most:

1. The pre-commitment rule

Decide — *in advance* — the conditions under which you will change or stop the system. For example:

- "If the drawdown exceeds X, I reduce size by half."
- "If live diverges from backtest by Y% for Z months, I stop and investigate."
- "Parameter changes happen only at a scheduled quarterly review, never mid-trade."

The point of pre-commitment is that these decisions are made in a **calm state**, before the drawdown or the euphoria arrives — because in the emotional state, the decision will be wrong.

2. The no-override rule

The system's outputs are followed **unless** a pre-committed condition (above) triggers. "I overrode it because it felt wrong" is the death of systematic trading — it is the moment the framework stops being systematic. Overrides are allowed only through the governance process, never in the moment.

3. The separation of roles

Where possible, separate the roles that conflict:

- The person who **generates** the signal should not be the one who **decides whether to follow it**.
- The person who **writes** the backtest should not be the only one who **reviews** it.

Even for a solo trader, *writing the rules down and treating them as binding* creates a crude separation between "designer me" (calm) and "operator me" (emotional) — which is the point.

The governance document

Governance lives in a written **governance document** — a short set of rules covering:

- **Change control** — when and how the system's rules may be changed.
- **Drawdown policy** — what happens at each drawdown level.
- **Stop conditions** — the kill switch (Chapter 8).
- **Review cadence** — how often the system is reviewed, and against what.

A system with a governance document is a *system*. Without one, it is a discretionary trader who occasionally uses a spreadsheet.

The honest bottom line

The framework's entire risk-and-governance apparatus exists for one reason: **to keep the operator from destroying the system**. The market will not destroy a well-sized, well-governed system quickly; the operator will — by over-betting in euphoria, abandoning in drawdown, or tweaking in panic. Governance is the firewall between the system and its own creator.

Summary

- Risk is baked into every layer as *rules*, not intentions.
- Governance is rules about the rules: who changes the system, and when.
- Pre-commit, no-override, and separation of roles are the three key governance rules.
- A written governance document is what makes a system a *system*.
- The framework's biggest risk is its own operator; governance is the firewall.

Next: the final chapter — a working framework template.

10. A Working Framework Template

Everything, on one page

The previous nine chapters built the framework piece by piece. This chapter assembles it into a **single working template** — the one-page document that turns the framework from an idea into a tool you can actually run.

The framework template

```
=====
SYSTEMATIC TRADING FRAMEWORK
=====

1. HYPOTHESIS
  Mechanism: [why the edge exists – behaviour/structure, in a sentence]
  Universe:  [exactly what is traded + liquidity filter]
  Signal:    [entry rule, in precise words]
  Exit:      [stop / target / trail / time rule]
  Claim:     [metric + benchmark + horizon]
  Falsify:   [result that would prove it wrong]

2. DATA
  Source:     [vendor / API, documented]
  Adjustment: [splits/bonuses/rights – how adjusted]
  Point-in-time: [yes – historical universe, un-restated fundamentals]
  Cleaning log: [what was dropped and why]

3. SIZING & PORTFOLIO
  Risk per trade: [1-2%]
  Size rule:      [(Equity × Risk%) ÷ (Entry – Stop)]
  Weighting:      [equal-weight / signal-weight]
  Max per position: [%]
  Max total at-risk: [%]
  Max exposure:   [%]
  Diversification: [across what drivers]

4. BACKTEST DISCIPLINE
  Costs included: [brokerage, STT, spread, slippage – pessimistic]
  Survivorship:   [removed – point-in-time universe]
  Look-ahead:     [hunted – "known on this date?" test]
  Parameter stress: [edge shown across a range, flat-topped]
  Out-of-sample:  [tested on unseen period]
  Sample:         [trade count, regime coverage]

5. EXECUTION & MONITORING
  Runbook:        [daily/weekly procedure]
  Reconciliation: [daily, planned vs. actual]
```

```
Slippage pad:      [%]
Live-vs-backtest: [tracked, deviation threshold]
```

6. GOVERNANCE

```
Change control:    [how/when rules change – scheduled review only]
Drawdown policy:   [action at each drawdown level]
Kill switch:       [conditions that stop the system]
Review cadence:    [how often, against what]
```

```
=====
```

How to use the template

1. **Fill it in before you build.** The template *forces* the thinking (mechanism, falsification, exit) that most people skip.
2. **Fill it in before you backtest.** Pre-registration — the hypothesis and the success/failure criteria are committed *before* the numbers arrive.
3. **Use it as the live runbook.** The execution, monitoring, and governance sections are your operating manual, not a formality.
4. **Review it on schedule.** The governance section defines *when* — and the discipline is to review then, and only then, not reactively.

The test of a working framework

A framework is "working" when it passes a single test: **you can hand the whole thing — the template, the data, the rules — to someone else, and they can run it and get the same decisions you would, without asking you a single question.**

If you have to explain, override, or interpret, the framework is not finished — it is discretionary trading wearing a spreadsheet. The entire discipline of this book reduces to that one test.

The honest closing

A framework does not guarantee profit. What it guarantees is something more valuable: **that your decisions are explicit, your testing is honest, and your failures are small enough to learn from.** That is the entire game — survive long enough, with a process clean enough, that a real edge (if you find one) can actually compound. Everything in this book, and in the whole series, serves that single goal.

Summary

- The template assembles hypothesis, data, sizing, backtest, execution, and governance on one page.
- Fill it in before building and before backtesting (pre-registration).

- It doubles as the live runbook and the governance document.
- The test: someone else can run it and get the same decisions, with no questions.
- The framework guarantees honesty and survival, not profit.

This completes the main text. The appendix is the lifecycle checklist.

Appendix — Lifecycle Checklist

Run this against any system you build

This checklist is the framework's lifecycle, condensed to a series of yes/no gates. A system is ready to proceed to the next stage **only when every item in the current stage is honestly "yes"**.

Stage 1 — Hypothesis

- ☐ I can state the *mechanism* (why the edge exists) in one sentence.
- ☐ The universe is specified exactly, with a liquidity filter.
- ☐ The signal (entry **and** exit) is written unambiguously.
- ☐ The success/failure criteria (metric, benchmark, horizon) are committed *before* testing.

Stage 2 — Data

- ☐ The data source is documented and auditable.
- ☐ Prices are adjusted for corporate actions (splits/bonuses/rights).
- ☐ The universe is point-in-time (includes delisted/failed names).
- ☐ Fundamentals (if used) are un-restated / point-in-time.
- ☐ A cleaning log records what was dropped and why.

Stage 3 — Backtest

- ☐ Costs are included, pessimistically (brokerage, STT, spread, slippage).
- ☐ Survivorship bias is removed.
- ☐ Look-ahead bias has been hunted ("known on this date?").
- ☐ Parameters are stressed (edge shown across a range, flat-topped).
- ☐ The edge survives an out-of-sample test.
- ☐ The sample is large enough, and spans multiple regimes.

Stage 4 — Paper trade

- ☐ The system runs live in real time with no (or tiny) money.
- ☐ Execution (fills, slippage) is reconciled against the model.
- ☐ Live costs match (or exceed) the backtest's estimate.
- ☐ The runbook and reconciliation process work without errors.

Stage 5 — Live

- [] Going live at small size first.
- [] Live performance is tracked against the backtest's prediction.
- [] Edge decay is monitored against a threshold.
- [] The kill switch and drawdown policy are in place and pre-committed.

Stage 6 — Review / retire

- [] The system is reviewed on the scheduled cadence (not reactively).
- [] Parameter/rule changes happen only through change control.
- [] The system is retired when it stops behaving as it should.
- [] The lessons are fed back into the next hypothesis.

The one-line test

Can someone else run the whole system — template, data, rules — and get the same decisions you would, without asking you a single question?

If the answer is yes, you have a framework. If not, you have a discretionary trader with a spreadsheet.